Build Task — Project 4 as an API — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Think of the Build Task as 3 layers, each one you've already built separately: the **API layer** (`chat_route` — typed request in, typed response out, 4xx/5xx handled correctly, real errors logged on the server only), the **storage layer** (`sqlite_persistence` + `validation_422` — but now with a `status` that changes as the run moves: `"started"` when the row is first written, `"completed"` or `"failed"` once it's done), and the **operations layer** (`db_failure_and_docker` — Dockerfile, secrets only via env vars, startup check — plus rate limiting).

Why status needs 3 states, not just "done": a task run can fail partway through — an agent might crash, an API call might time out. Saving a `"started"` row *before* running the task (in its own `with conn:`), then updating it to `"completed"` or `"failed"` after, means a crashed run still leaves a real, honest record. That's the one deliberate difference from `validation_422` , which rolled both writes back together.

Why the request ID matters here specifically: a multi-agent run writes many log lines per request. Without a shared ID on each, you can't tell which lines belong to which request once two people call your API at the same time.

The honest limitation of an in-memory rate limiter (the Requirements ask you to write it down): a Python dict forgets everything when the process restarts, and if you run more than one copy of the API (which real deployments do), each copy has its *own* dict — so a caller could get 3x

their limit by landing on 3 copies. The real fix is a shared store all copies can see (Redis is the standard choice). Put this in a comment right next to the dict.

Keeping secrets out of logs: never log the API key, request headers, or `str(exc)` from a provider SDK error without looking at it first. `logger.exception()` with your own short message, as in `chat_route`, is the safe default.

The exact pieces:

- `api/schemas.py` → `TaskRequest(task: str)`, `TaskResponse(run_id, result, status)` — like `chat_schemas.py`.
- `db/database.py` → `get_connection()`, `insert_started_run()`, `finish_run()`, `fail_run()`, `fetch_run()` — like `runs_db.py`, each write in its own `with conn:`.
- `db/models.py` → a `Run(BaseModel)` for `GET /runs/{id}`'s `response_model`; `output` and `log` are `str | None`, since they're empty while a run is `"started"` or `"failed"`.
- `api/routes.py` → an `APIRouter` with the 3 routes, plus `check_rate_limit(caller_key)`, called with `http_request.client.host` (the caller's IP address).
- `api/main.py` → the startup env check, `app.include_router(router)`, and the request-ID middleware.
- `"\n".join(steps)` — turns the agent's list of steps into one text value for the `log` column.
- `test_api.py` → `TestClient`, one check per row of the Test Cases table, printed with `# expected:` comments — like `health_route`'s.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
split into the suggested files:
    api/main.py, api/routes.py, api/schemas.py
    db/database.py, db/models.py
    logging_setup.py (copied from Doc01, unchanged)

db/database.py: get_connection, insert_started_run, finish_run,
                fail_run, fetch_run — each write inside its own "with conn:"

api/main.py: startup env check (MissingConfigError), app,
             include_router, request-ID middleware

api/routes.py:
    rate limiting: a dict of {caller: [timestamp, timestamp, ...]}
        on each request: keep only timestamps inside the window, then:
            if len(recent) >= limit: raise 429
```

```
            else: add this request's timestamp, carry on
    every log line starts with "[request_id]"
```

Here's the rate limiter — fill in the loop yourself:

```
import time
from fastapi import HTTPException

RATE_LIMIT = 5             # max requests per caller...
RATE_WINDOW_SECONDS = 60   # ...inside this many seconds
# LIMITATION: write the honest limitation here, in your own words
request_times = {}         # caller -> list of request timestamps

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    old_times = request_times.get(caller_key, [])
    recent = []
    # your turn: loop over old_times, append the ones where
    # now - t < RATE_WINDOW_SECONDS to recent
    ...
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    recent.append(now)
    request_times[caller_key] = recent
```

Expected output for the 6th request within 60 seconds from the same caller:

```
429 {"detail":"Rate limit exceeded"}
```

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

A small `logging_setup.py` with `get_logger()` (from Doc01) is included in `practice/build_task/`, so the service runs on its own.

Story — `db/database.py`: every SQL statement lives here, behind small named functions, like `runs_db.py` in `sqlite_persistence`. Each write gets its own `with conn:`, so the "started" row is saved on its own and a crash later leaves an honest "failed" record. **If not:** SQL would be scattered through the routes, and a crashed run could leave either nothing or a row stuck at "started" forever.

```
# practice/build_task/db/database.py
import sqlite3
from datetime import datetime, timezone

# why: the Build Task's one table — same columns as sqlite_persistence,
# plus log and status
CREATE_RUNS_TABLE = """
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        log TEXT,
        status TEXT,
        created_at TEXT
    )
"""

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute(CREATE_RUNS_TABLE)
    return conn

def insert_started_run(conn, task_input):
    now = datetime.now(timezone.utc).isoformat()
    # why: its own transaction — the "started" row is saved right away,
    # so a crash later still leaves a record to look up
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (task_input, "started", now),
        )
    return cursor.lastrowid

def finish_run(conn, run_id, output, log_text):
    with conn:
        conn.execute(
            "UPDATE runs SET output = ?, log = ?, status = ? WHERE id = ?",
            (output, log_text, "completed", run_id),
        )

def fail_run(conn, run_id):
    with conn:
        conn.execute(
            "UPDATE runs SET status = ? WHERE id = ?",
            ("failed", run_id),
        )
```

```
def fetch_run(conn, run_id):
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    # when: no row with this id — the route turns None into a 404
    if row is None:
        return None
    return dict(row)
```

Story — `db/models.py` and `api/schemas.py`: the shapes going in and out of the API, written down once, like `RunRecord` and `chat_schemas.py`. **If not:** `GET /runs/{id}` could leak any column you add to the table later, and a wrong field name would only be found by the client.

```
# practice/build_task/db/models.py
from pydantic import BaseModel

# why: one saved run's shape, checked on the way out —
# same idea as RunRecord in sqlite_persistence
class Run(BaseModel):
    id: int
    input: str
    output: str | None     # None while the run is "started" or "failed"
    log: str | None
    status: str
    created_at: str
```

```
# practice/build_task/api/schemas.py
from pydantic import BaseModel

class TaskRequest(BaseModel):
    task: str

class TaskResponse(BaseModel):
    run_id: int
    result: str
    status: str
```

Story — `api/routes.py`: the three routes on an `APIRouter` (like `health_routes.py`), plus the rate limiter. `POST /run-task` is `chat_route`'s shape with a database wrapped around it: the same 400/500 split, the same `logger.exception`, the same `[request_id]` at the start of every log line. **If not:** a failed run would leave no trace in the logs, and one caller could run up your OpenAI bill with no limit at all.

```
# practice/build_task/api/routes.py
import time
from fastapi import APIRouter, HTTPException, Request

from api.schemas import TaskRequest, TaskResponse
from db.database import get_connection, insert_started_run
from db.database import finish_run, fail_run, fetch_run
from db.models import Run
from logging_setup import get_logger   # Doc01 Build Task

logger = get_logger(__name__)
router = APIRouter()
conn = get_connection()

RATE_LIMIT = 5             # max requests per caller...
RATE_WINDOW_SECONDS = 60   # ...inside this many seconds
# LIMITATION: in-memory only. It resets on every restart, and each
# worker/container keeps its own separate counts — so 3 containers
# really allow 3 x RATE_LIMIT. A shared store (like Redis) fixes that.
request_times = {}         # caller -> list of request timestamps

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    old_times = request_times.get(caller_key, [])
    recent = []
    for t in old_times:
        # how: keep only the timestamps still inside the window
        if now - t < RATE_WINDOW_SECONDS:
            recent.append(t)
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    recent.append(now)
    request_times[caller_key] = recent

def run_project_4_task(task_input: str):
    # stand-in for Project 4's real graph — swap in the real one
    # why: a fake crash on command, same trick as chat_route's "boom"
    if task_input == "boom":
        raise RuntimeError("simulated internal failure")
    steps = [
        "supervisor -> research_agent",
        "supervisor -> writer_agent",
        "supervisor -> reviewer_agent: approved",
    ]
    return "result for: " + task_input, steps

@router.get("/health")
```

```
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest, http_request: Request) -> TaskResponse:
    request_id = http_request.state.request_id
    check_rate_limit(http_request.client.host)
    # why: "" passes Pydantic (it IS a str) — our own 400, like chat_route
    if not request.task.strip():
        raise HTTPException(status_code=400, detail="task cannot be empty")

    run_id = insert_started_run(conn, request.task)
    logger.info("[%s] run %s started", request_id, run_id)
    try:
        output, steps = run_project_4_task(request.task)
    except Exception:
        fail_run(conn, run_id)
        # how: full traceback in the server log only
        logger.exception("[%s] run %s failed", request_id, run_id)
        raise HTTPException(
            status_code=500, detail="Internal error. Please try again later."
        )

    log_text = "\n".join(steps)
    finish_run(conn, run_id, output, log_text)
    logger.info("[%s] run %s completed", request_id, run_id)
    return TaskResponse(run_id=run_id, result=output, status="completed")

@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id: int):
    run = fetch_run(conn, run_id)
    if run is None:
        raise HTTPException(status_code=404, detail="run not found")
    return run
```

Story — `api/main.py`: the one file uvicorn starts. It checks the required setting first, then builds the app, adds the routes, and adds the request-ID middleware from `chat_route`. **If not:** a container started without its key would look healthy and fail on every real request, and log lines from two requests would be impossible to tell apart.

```
# practice/build_task/api/main.py
import os
import uuid
from fastapi import FastAPI, Request

from api.routes import router
```

```
class MissingConfigError(Exception):
    """Same custom error as Doc01's exceptions.py."""

# why: fail at startup, not on the first real request
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    raise MissingConfigError(
        "OPENAI_API_KEY is not set — pass it with docker run -e"
    )

app = FastAPI()
app.include_router(router)

# why: one short ID per request, made in one place for every route
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

Expected output (`OPENAI_API_KEY=sk-test-123 uvicorn api.main:app` , then the same `curl` as Basic):

```
HTTP/1.1 200 OK
x-request-id: a1b2c3d4
{"run_id":1,"result":"result for: summarize this","status":"completed"}
```

Server log for that request (both lines share the same ID):

```
2026-01-01 00:00:00,000 api.routes INFO [a1b2c3d4] run 1 started
2026-01-01 00:00:00,002 api.routes INFO [a1b2c3d4] run 1 completed
```